SHOPSPHERE ONLINE E-COMMERCE DATABASE MANAGEMENT SYSTEM

WEEK 6 – PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

1. INTRODUCTION

The Product Review and Rating Management System is a module of the ShopSphere Online E-Commerce Database Management System. It allows customers to provide feedback and ratings for products they purchase. The system stores customer reviews, ratings, review dates, and product details. This module helps ShopSphere identify highly rated products and calculate the average rating of each product using SQL aggregate functions.

2. OBJECTIVES

The main objectives of the Product Review and Rating Management System are:

- To create Review and Rating tables.
- To store customer feedback and product ratings.
- To retrieve product review details.
- To calculate the average rating of each product.
- To identify highly rated products.
- To improve product evaluation using customer feedback.

3.1 Review Table

Column	Data Type	Description
Review_ID	NUMBER	Unique ID for each review
Customer_ID	NUMBER	ID of the customer
Product_ID	NUMBER	ID of the product
Review_Text	VARCHAR2(500)	Customer feedback
Review_Date	DATE	Date of the review

3.2 Rating Table

Attribute	Data Type	Constraint
Rating_ID	INT	Primary Key
Customer_ID	INT	Foreign Key
Product_ID	INT	Foreign Key
Rating	INT	NOT NULL
Rating_Date	DATE	NOT NULL

4. CREATE REVIEW TABLE

The Review table is created to store customer feedback.

```
SQL> CREATE TABLE Review (
  2      Review_ID NUMBER PRIMARY KEY,
  3      Customer_ID NUMBER,
  4      Product_ID NUMBER,
  5      Review_Text VARCHAR2(500),
  6      Review_Date DATE DEFAULT SYSDATE,
  7      FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
  8      FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
  9  );

Table created.
```

5. CREATE RATING TABLE

The Rating table stores ratings from 1 to 5.

```
SQL> CREATE TABLE Rating (
  2      Rating_ID INT PRIMARY KEY,
  3      Customer_ID INT,
  4      Product_ID INT,
  5      Rating INT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
  6      Rating_Date DATE NOT NULL,
  7      FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
  8      FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
  9  );

Table created.
```

6. INSERT CUSTOMER REVIEWS

```
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3  VALUES
  4  (701, 101, 101, 'Excellent product and good quality', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3  VALUES
  4  (702, 102, 102, 'Very useful and worth the price', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3  VALUES
  4  (703, 103, 101, 'Good product but delivery was late', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3  VALUES
  4  (704, 104, 103, 'Amazing quality and performance', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2  (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date)
  3  VALUES
  4  (705, 105, 104, 'Average product', SYSDATE);

1 row created.
```

7. INSERT PRODUCT RATINGS

```
SQL> INSERT INTO Rating
  2  (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
  3  VALUES
  4  (801, 101, 101, 5, SYSDATE);

1 row created.

SQL> INSERT INTO Rating
  2  (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
  3  VALUES
  4  (802, 102, 102, 4, SYSDATE);

1 row created.

SQL> INSERT INTO Rating
  2  (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
  3  VALUES
  4  (803, 103, 101, 3, SYSDATE);

1 row created.

SQL> INSERT INTO Rating
  2  (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
  3  VALUES
  4  (804, 104, 103, 5, SYSDATE);

1 row created.

SQL> INSERT INTO Rating
  2  (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date)
  3  VALUES
  4  (805, 105, 104, 3, SYSDATE);

1 row created.
```

8. RETRIEVE PRODUCT REVIEW DETAILS

The following query retrieves the review details along with the rating.

```
SQL> SELECT
  2      r.Review_ID,
  3      r.Customer_ID,
  4      r.Product_ID,
  5      r.Review_Text,
  6      r.Review_Date,
  7      rt.Rating
  8  FROM Review r
  9  JOIN Rating rt
 10  ON r.Review_ID = rt.Review_ID;

 REVIEW_ID CUSTOMER_ID PRODUCT_ID
---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA     RATING
--------- ----------
       701         101        101
Excellent product and good quality
06-OCT-26          5

       702         102        102
Very useful and worth the price
06-OCT-26          4

 REVIEW_ID CUSTOMER_ID PRODUCT_ID
---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA     RATING
--------- ----------
```

```
       703         103        101
Good product but delivery was late
06-OCT-26          3

       704         104        103
Amazing quality and performance

 REVIEW_ID CUSTOMER_ID PRODUCT_ID
---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA     RATING
--------- ----------
06-OCT-26          5

       705         105        104
Average product
06-OCT-26          3
```

10. CALCULATE AVERAGE PRODUCT RATINGS

The AVG() and COUNT() aggregate functions are used to calculate the average rating and total number of ratings for each product.

```
SQL> SELECT
  2      Product_ID,
  3      ROUND(AVG(Rating), 2) AS Average_Rating,
  4      COUNT(Rating) AS Total_Ratings
  5  FROM Rating
  6  GROUP BY Product_ID;

PRODUCT_ID AVERAGE_RATING TOTAL_RATINGS
---------- -------------- -------------
       101              4             2
       102              4             1
       103              5             1
       104              3             1
```

12. IDENTIFY HIGHLY RATED PRODUCTS

Products with an average rating of 4 or above are considered highly rated.

```
SQL> SELECT
  2      Product_ID,
  3      ROUND(AVG(Rating), 2) AS Average_Rating
  4  FROM Rating
  5  GROUP BY Product_ID
  6  HAVING AVG(Rating) >= 4;

PRODUCT_ID AVERAGE_RATING
---------- --------------
       101              4
       102              4
       103              5
```

15. CONCLUSION

The Product Review and Rating Management System was successfully implemented as part of the ShopSphere Online E-Commerce Database Management System. The system stores customer feedback and ratings, retrieves review information, calculates average product ratings, and identifies highly rated products. This module helps ShopSphere understand customer satisfaction and evaluate product quality efficiently.